# 章末問題 解答

## 第 1 章

### 問 1.1

プログラム p1-1 を実行すると以下の値が出力されます．

```
 x = inf
 s |        e        |                    f
---+-----------------+-------------------------------------------------------------
 0 | 111 1111 1111   | 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

 x = 8.28904605845809498090e-317
 s |        e        |                    f
---+-----------------+-------------------------------------------------------------
 0 | 000 0000 0000   | 0000 0000 0000 0000 0000 0000 0001 0000 0000 0000 0000 0000 0000
```

上記から，指数部は無限大のとき 1024, 非正規化数のとき-1023 となっており，正規化数の指数の範囲外となっていることがわかります．

### 問 1.2

倍精度浮動小数点数では仮数の桁数が 53 桁なので，1 刻みで表現できる数の最大値は $2^{53}$( = 9007199254740992) となります．したがってこの値から 1 を引いた 9007199254740991 が正しく計算できる最大の数となります．実際に計算して確認するプログラムを p1-2.cpp に示します．

### 問 1.3

通常の総和計算とカハンのアルゴリズムによる総和計算を行うプログラムを p1-3.cpp に示します．実行結果は

カハンのアルゴリズム ： 1.6449329853057861
通常の総和計算　　： 1.6449329853057861

となり，カハンのアルゴリズムによる効果は見られません．理由としては，計算途中における総和と誤差の値の差が大きいため，誤差を累積したとしても再度総和にくり込むことができないためです．したがってこのようなケースではカハンのアルゴリズムは上手く機能しないことになります．

### 問 1.4

問題文中のプログラムにおいて，ループ処理ごとに変数 $x$ の値を出力すると以下のようになります．

x = 0.699999999999999956
x = 1.399999999999999911

x = 2.099999999999999645
x = 2.799999999999999822
x = 3.500000000000000000
x = 4.200000000000000178
x = 4.900000000000000355
x = 5.600000000000000533
x = 6.300000000000000711
x = 7.000000000000000888

最後の $x$ の値は本来 7.0 となるべきところ，計算誤差によって 7.000$\cdots$00888 となっています．そのため while 文の継続条件 $x <= 7.0$ が成立せず，正しく計算できた場合に比べ 1 回早くループを抜けています．

この点を修正し，式（1.17）の通りに計算するプログラムを p1-4.cpp に示します．

## 第 2 章

**問 2.1**

(条件 1) 1 ノルムの定義式から明らか．

(条件 2)

$$\|\alpha \boldsymbol{x}\|_1 = \sum_{i=1}^{n} |\alpha x_i| = |\alpha| \sum_{i=1}^{n} |x_i| = |\alpha| \|\boldsymbol{x}\|_1 \tag{1}$$

(条件 3)

$$\|\boldsymbol{x} + \boldsymbol{y}\|_1 = \sum_{i=1}^{n} |x_i + y_i| \leq \sum_{i=1}^{n} |x_i| + \sum_{i=1}^{n} |y_i| = \|\boldsymbol{x}\|_1 + \|\boldsymbol{y}\|_1 \tag{2}$$

**問 2.2**

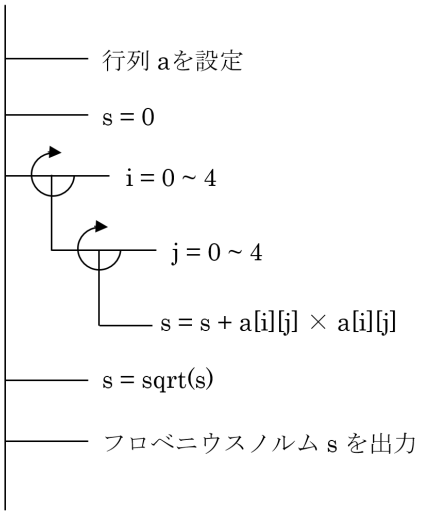
プログラムを p2-2.cpp に示します．

**問 2.3**

プログラムを p2-3.cpp に示します．

**問 2.4**

SPD を以下に，プログラムを p2-4.cpp に示します．




## 第 3 章

**問 3.1**

ガウスの消去法で解いた場合,

$$
\begin{aligned}
x_1 &= 1.000000000232420 \\
x_2 &= -1.000000000006524 \\
x_3 &= -2.000000000000000 \\
x_4 &= 2.000000000000078
\end{aligned}
$$

となり，ピボット選択付きガウスの消去法で解いた場合は

$$
\begin{aligned}
x_1 &= 1.000000000000025 \\
x_2 &= -1.000000000000025 \\
x_3 &= -2.000000000000019 \\
x_4 &= 2.000000000000050
\end{aligned}
$$

となります．若干ですがピボット選択付きガウスの消去法の方が精度が良いことが分かります．

**問 3.2**

問題文の対偶を示します．まず $n$ 次の係数行列を $A = [a_{ij}]$ とします．部分ピボット選択付きガウスの消去法で解が求まらないケースには以下の 2 つがあります．

(1) 前進消去の処理中ピボット選択において，どの行と入れ換えを行っても対角成分が 0 になってしまう場合

(2) 前進消去終了後，$a_{nn}$ が 0 になる場合

まず (1) の場合を考えます．今, 第 $i$ 列の前進消去においてそのような状況になったとします．このときの係数行列を $A'$ とすると，$A'$ の第 $i$ 列の対角成分以下は全て 0 です．

ここで前進消去の操作は，ランクを求めるために行基本変形を用いて階段行列に変形する操

作と同等です．また，第 $i$ 列の対角成分以下が全て 0 である行列 $A'$ に対して行基本変形を継続し，階段行列に変形するとその最下行 ($n$ 行目) はすべて 0 となります．

以上のことから $A$ のランクは $n$ 未満となり，正則ではないことが分かります．また (2) のケースでは $A$ のランクは $n-1$ であり，正則ではありません．したがって係数行列が正則であれば，ピボット選択付きガウスの消去法により必ず解を求めることができます．

なお，逆に部分ピボット選択付きガウスの消去法で解が求まった場合，前進消去後の行列のランクは $n$ であることから $A$ の正則性がいえます．したがって $A$ が正則であることと，ガウスの消去法で解が求まることは同値であることが分かります．

**問 3.3**

プログラムを p3-3.cpp に示します．四則演算の回数は $N^3+N^2-N$ 回となります．したがって，本文で紹介したガウスの消去法の方が計算時間は短くなります．

**問 3.4**

プログラムを p3-4.cpp に示します．

**問 3.5**

演算回数は $3N^3-2N^2$ 回になります．

## 第 4 章

**問 4.1**

$n$ 次元単位ベクトル $\boldsymbol{e}_i$ を，第 $i$ 成分のみ 1 でその他の成分は 0 であるベクトルとします．$n$ 個の方程式 $A\boldsymbol{x}_i=\boldsymbol{e}_i$ $(i=1,2,\cdots,n)$ の解 $\boldsymbol{x}_i$ に対し，各 $\boldsymbol{x}_i$ を第 $i$ 列とするような行列 $B$ を考えると，$B$ は $A$ の逆行列となります．このことを用いたプログラムを p4-1.cpp に示します．

**問 4.2**

演算回数は $\frac{8}{3}N^3-\frac{1}{2}N^2-\frac{7}{6}N$ 回です．したがって N が大きくなると問 3.3 の掃き出し法より演算回数は少なくなります．

**問 4.3**

【問題訂正】

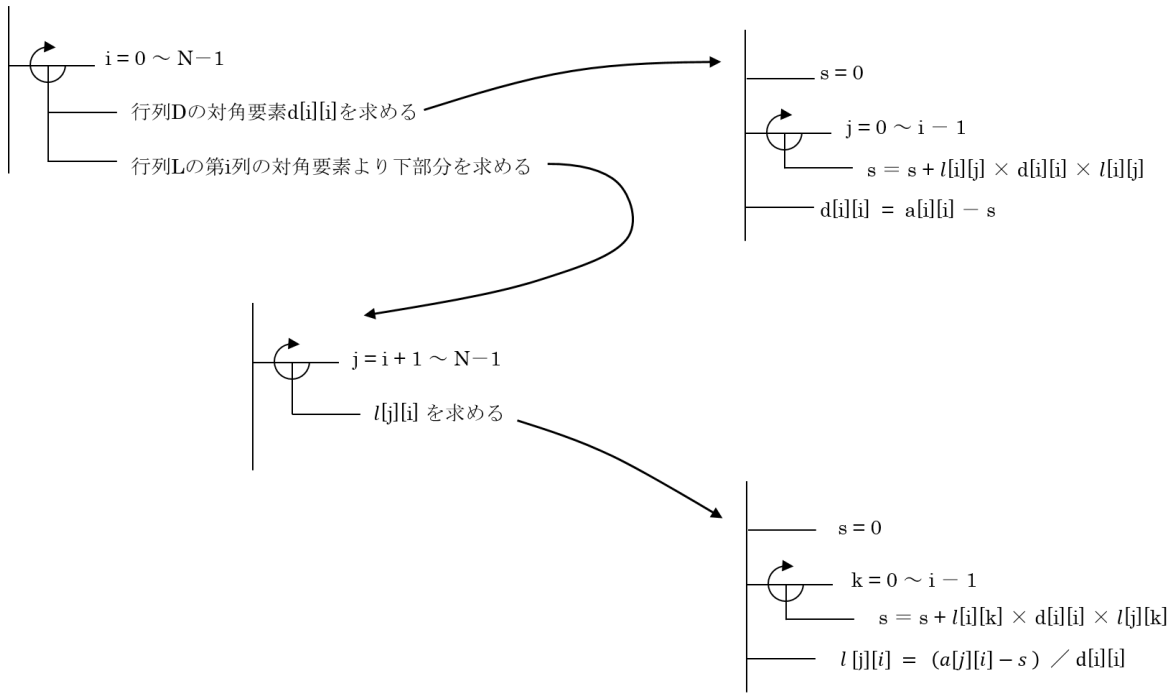
正誤表に記載したように式 (4.41) の係数行列が正定値でないため，係数行列を以下のように訂正した上で解答を示します．

$$\begin{bmatrix} 15 & 7 & -6 & 8 \\ 7 & 6 & -7 & 3 \\ -6 & -7 & 14 & 1 \\ 8 & 3 & 1 & 13 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -2 \\ 6 \\ -3 \\ 1 \end{bmatrix} \tag{3}$$

上式に対する解答をプログラム p4-3.cpp に示します．

**問 4.4**

SPD を以下に示します．




## 第 5 章

**問 5.1**

プログラムを p5-1.cpp に示します．

**問 5.2**

式 (5.74) の方程式に対して，以下のように行を入れ替えると収束します．

1 行目： 式 (5.74) の 2 行目

2 行目： 式 (5.74) の 4 行目

3 行目： 式 (5.74) の 1 行目

4 行目： 式 (5.74) の 3 行目

**問 5.3**

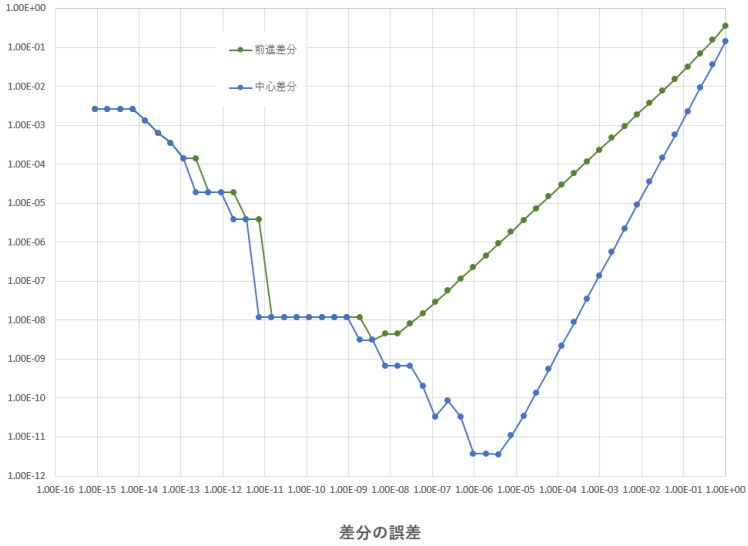
100 元や 1000 元の方程式に対して共役勾配法を実行した場合，必ずしも 100 回または 1000 回の反復は必要ありません．問題にもよりますが，多くの場合数回～数十回程度の反復で収束します．

## 第 6 章

**問 6.1**

$n = 1, 2, \cdots, 50$ とした場合の前進差分，中心差分の誤差を以下に示します．最初 $h$ が小さくな

るにしたがって誤差は減少しますが，途中から再び誤差が大きくなる様子が確認できます．




**問 6.2**

プログラムを p6-2.cpp に示します．

プログラムの実行結果 ： $\tan(0.5) = 0.546302$，$\tan'(0.5) = 1.298446$

**問 6.3**

計算グラフの各ノードにおいて，そのノードより下に繋がっているノードの中に変数が存在しない場合は，そこから下のノードに対する微分値の計算は行わないようにします．具体的には，backward 関数において，上記条件が成立する場合は子ノードに対する再帰を行わないようプログラムを修正します．

修正したプログラム p6-3.cpp ではこの条件の成立を確認するために，各ノードの下に変数が存在するか否かを示すフラグを Node クラスのデータメンバとして追加しています．フラグの変数名を v_flag とし，値のセットは以下のように行います．

(1) Node 型の変数（例えば x とする）を宣言する際に，x の v_flag を true にセットする．
(2) 演算（四則演算や初等関数）を行う際，引数のノードの v_flag が true であれば演算結果のノードの v_flag も true にする

このようにすることで計算グラフが完成した際，各ノードの v_flag が正しくセットされ，子ノードに変数が含まれるか否か判断できます．

## 第 7 章

**問 7.1**

プログラムを p7.1.cpp に示します．

**問 7.2**

プログラムを p7.2.cpp に示します．

**問 7.3**

$f(x)=0$ の解を $\alpha$ とし，$f(x_n)$ にテイラーの定理を適用すると

$$\begin{aligned} f(x_n) &= f(\alpha+(x_n-\alpha)) \\ &= f(\alpha)+f'(\alpha)(x_n-\alpha)+\frac{f''(\xi_n)}{2}(x_n-\alpha)^2, \quad (\alpha<\xi<x_n \text{ または } x_n<\xi<\alpha) \\ &= f'(\alpha)(x_n-\alpha)+\frac{f''(\xi_n)}{2}(x_n-\alpha)^2 \end{aligned} \tag{4}$$

となります．また，上式と式 (7.48) より

$$\begin{aligned} x_{n+1}-\alpha &= x_n-\alpha-\frac{f(x_n)}{f'(x_0)} \\ &= (x_n-\alpha)-\frac{1}{f'(x_0)}\left\{f'(\alpha)(x_n-\alpha)+\frac{f''(\xi_n)}{2}(x_n-\alpha)^2\right\} \\ &= \left\{\frac{f'(x_0)-f'(\alpha)}{f'(x_0)}-\frac{f''(\xi_n)}{2f'(x_0)}(x_n-\alpha)\right\}(x_n-\alpha) \end{aligned} \tag{5}$$

ここで $x_0\simeq\alpha$ とすると

$$\left|\frac{f'(x_0)-f'(\alpha)}{f'(x_0)}\right|<1 \tag{6}$$

また

$$\frac{f''(\xi_n)}{2f'(x_0)}(x_n-\alpha)\to 0 \quad (n\to\infty) \tag{7}$$

より，線形収束となることがわかります．

## 第 8 章

**問 8.1**

$\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{0}$ の解を $\boldsymbol{\alpha}$ とし，テイラーの定理を適用すると

$$\begin{aligned} 0=f_k(\boldsymbol{\alpha}) &= f_k(\boldsymbol{x}^{(m)})+\sum_{i=1}^{n}\frac{\partial f_k}{\partial x_i}(\boldsymbol{x}^{(m)})(\alpha_i-x_i^{(m)}) \\ &\quad +\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\frac{\partial f_k^2(\boldsymbol{x}^{(m)}+\theta_k(\boldsymbol{\alpha}-\boldsymbol{x}^{(m)}))}{\partial x_i\partial x_j}(\alpha_i-x_i^{(m)})(\alpha_j-x_j^{(m)}) \\ &\qquad\qquad (0<\theta_k<1) \end{aligned} \tag{8}$$

上式の右辺第 3 項を第 $k$ 成分とするベクトルを $\boldsymbol{v}$ とおき，上式の両辺に $J(\boldsymbol{x}^{(m)})^{-1}$ をかけると

$$0=J(\boldsymbol{x}^{(m)})^{-1}\boldsymbol{f}(\boldsymbol{x}^{(m)})+\boldsymbol{\alpha}-\boldsymbol{x}^{(m)}+J(\boldsymbol{x}^{(m)})^{-1}\boldsymbol{v} \tag{9}$$

ここで，十分大きな $m$ において

$$\left|\frac{\partial f_k^2(\boldsymbol{x}^{(m)}+\theta_k(\boldsymbol{\alpha}-\boldsymbol{x}^{(m)}))}{\partial x_i\partial x_j}\right|\le M \tag{10}$$

とすると

$$
\begin{aligned}
\left\|\boldsymbol{x}^{(m+1)}-\boldsymbol{\alpha}\right\|_\infty &= \left\|\boldsymbol{x}^{(m)}-\boldsymbol{\alpha}-J(\boldsymbol{x}^{(m)})^{-1}\boldsymbol{f}(\boldsymbol{x}^{(m)})\right\|_\infty \\
&= \left\|J(\boldsymbol{x}^{(m)})^{-1}\boldsymbol{v}\right\|_\infty \\
&\leq \left\|J(\boldsymbol{x}^{(m)})^{-1}\right\|_\infty \|\boldsymbol{v}\|_\infty \\
&\leq \frac{n^2}{2}M\left\|J(\boldsymbol{x}^{(m)})^{-1}\right\|_\infty \left\|\boldsymbol{x}^{(m)}-\boldsymbol{\alpha}\right\|_\infty^2 \qquad (11)
\end{aligned}
$$

となり，ニュートン法は2次収束することが分かります．

**問 8.2**

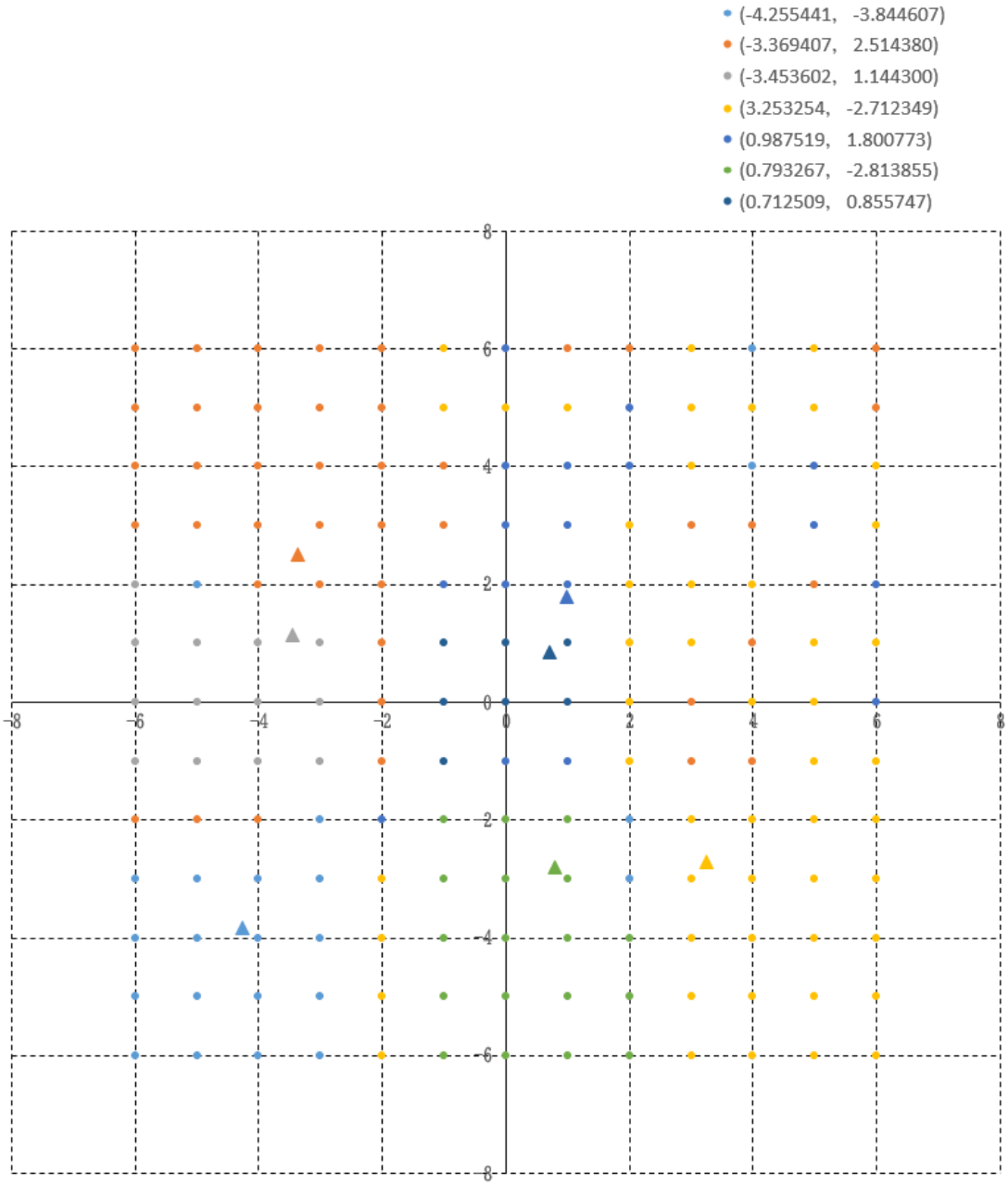
プログラムをp8-2.cppに示します．なお，本文中のニュートン法のプログラム「ソースコード8.2」では初期値としてベクトル$(1,1,1)$を用いましたが，簡易ニュートン法の場合，この初期値では収束しないためここでは初期値$(1,1,2)$を用いています．また，これにあわせてニュートン法も同じ初期値を用いて両者の比較を行うと，次のようになります．

| | ニュートン法 | 簡易ニュートン法 |
|---|---|---|
| 反復回数 | 5回 | 44回 |
| 実行時間 | 約0.000013秒 | 約0.000012秒 |

簡易ニュートン法では，ヤコビ行列の逆行列を更新しないため反復回数は多くなりますが，1回の反復については，行列とベクトルの乗算だけで近似解の修正量が求まるため，計算量は少なくなります．そのため，この問題におけるトータルの実行時間に関して大きな差はありません（なお，問題の次元数や非線形性の強さ，また初期値をどこに取るか等によって結果は異なります）．

**問 8.3**

プログラムをp8-3.cppに示します．このプログラムでは，初期値を変更しながらニュートン法を実行し，どの解に収束したかを出力しています．また，どの初期値からどの解に収束したかを図示するために，Excelを用いて散布図を作成する元ファイルをcvs形式で出力しています．出力されたファイルをもとに収束状況をプロットした図が以下になります．概ね初期点に近い解に収束していることがわかります．




## 第 9 章

**問 9.1**

プログラムを p9-1.cpp に示します．なお閉円板の中心は全て実軸上のため，実部のみを出力しています．また複素数を要素にもつ行列を対象とする場合も p9-1 とほぼ同様に作成できます．

**問 9.2**

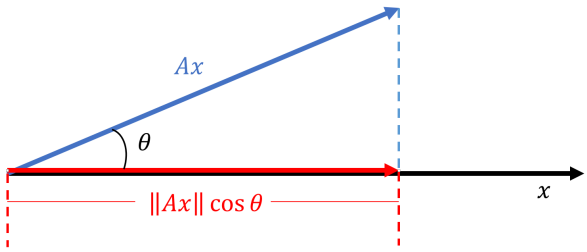
レイリー商の定義を以下に示します．

$$R(\boldsymbol{x}) = \frac{(\boldsymbol{x}, A\boldsymbol{x})}{(\boldsymbol{x}, \boldsymbol{x})} \tag{12}$$

下図に示すように，ベクトル $A\boldsymbol{x}$ と $\boldsymbol{x}$ のなす角を $\theta$ とすると

$$R(\boldsymbol{x}) = \frac{\|\boldsymbol{x}\|\|A\boldsymbol{x}\|\cos\theta}{\|\boldsymbol{x}\|^2} = \frac{\|A\boldsymbol{x}\|\cos\theta}{\|\boldsymbol{x}\|} \tag{13}$$

となります．




ここで固有値とは，固有ベクトル $\boldsymbol{x}$ に対し，行列 $A$ をかけることでその方向（固有ベクトルの方向）に何倍拡大（または縮小）されるかを表しています．

$x$ が固有ベクトルでない場合，$A\boldsymbol{x}$ は $\boldsymbol{x}$ に対して平行とはなりませんが，図のように $A\boldsymbol{x}$ を $x$ に射影し $\frac{\|A\boldsymbol{x}\|\cos\theta}{\|\boldsymbol{x}\|}$ を計算することで，$A\boldsymbol{x}$ が $\boldsymbol{x}$ の方向に何倍されているかを求めることができます．この意味でレイリー商は固有値の近似と見なせます．

**問 9.3**

式 (9.43)（注：式の訂正あり．正誤表参照）の左辺の $a_{ij}$ の添え字を $k$ とすると

$$\sum_{i\neq j} {a_{ij}^{(k)}}^2 \leq \left(1 - \frac{2}{(n^2-n)}\right)^k \sum_{i\neq j} {a_{ij}^{(0)}}^2 \tag{14}$$

となります．上式の右辺に対し

$$\left(1 - \frac{2}{(n^2-n)}\right)^k \sum_{i\neq j} {a_{ij}^{(0)}}^2 \leq \varepsilon \tag{15}$$

とおき，両辺の対数をとると

$$k\log\left(1 - \frac{2}{(n^2-n)}\right) + \log\left(\sum_{i\neq j} {a_{ij}^{(0)}}^2\right) \leq \log\varepsilon$$

$$k \leq \frac{\log\varepsilon - \log\left(\sum_{i\neq j} {a_{ij}^{(0)}}^2\right)}{\log\left(1 - \frac{2}{(n^2-n)}\right)} \tag{16}$$

となります．$k$ は整数であることを考慮すると反復回数の上限は

$$\left\lceil \frac{\log\varepsilon - \log\left(\sum_{i\neq j} {a_{ij}^{(0)}}^2\right)}{\log\left(1 - \frac{2}{(n^2-n)}\right)} \right\rceil \tag{17}$$

となります．

## 第 10 章

### 問 10.1

数学的帰納法により行列 $Q$ がヘッセンベルグ行列であることを示します．まず以下の条件を満たす行列 $G_k = [g_{ij}^{(k)}]$ $(k = 1, 2, \cdots, n-1)$ を考えます．

［条件 1］$j = 1, 2, \cdots, k+1$ において

$$g_{i,j}^{(k)} = 0 \quad (i > j+1) \tag{18}$$

［条件 2］$j = k+2, k+3, \cdots, n$ において

$$g_{i,j}^{(k)} = \begin{cases} 1 & (i = j) \\ 0 & (i \neq j) \end{cases} \tag{19}$$

このとき $G_k$ に $P_{k+1}^t$ を右から掛けた行列を $G_{k+1}$ とすると，$G_{k+1}$ は，$G_k$ の第 $k+1$ 列と第 $k+2$ 列の値のみが変更されたものとなります．さらに上の条件を考慮して $G_k P_{k+1}$ を求めると，その第 $k+1$ 列，第 $k+2$ 列は，

$$g_{i,k+1}^{(k+1)} = 0 \quad (i > k+2) \tag{20}$$

$$g_{i,k+2}^{(k+1)} = 0 \quad (i > k+2) \tag{21}$$

となります．この結果から $G_{k+1}$ は，添え字 $k$ を 1 ずらし $k = k+1$ とおいた場合，条件 1，2 を満たすことが分かります．

また $G_1 = P_1^t$ とすると $G_1$ は上の条件を満足することから，帰納法によって $G_{n-2}$ も条件 1，2 を満足します．さらに $G_{n-1} = G_{n-2} P_{n-1} = Q$ を計算すると $G_{n-1}$ のすべての列において $g_{ij}^{(n-1)} = 0$ $(i > j+1)$ となることがわかります．したがって $Q$ はヘッセンベルグ行列となります．

次に $RQ$ もヘッセンベルグ行列となることを示します．まず，$R$ の第 $i$ 行からなる行ベクトルを $\boldsymbol{r}^{(i)}$，$Q$ の第 $j$ 列からなる列ベクトルを $\boldsymbol{q}^{(j)}$ とします．$R$ は上三角行列，$Q$ はヘッセンベルグ行列であることから $r_k^{(i)} = 0$ $(k < i)$，$q_k^{(j)} = 0$ $(k > j+1)$ となります．ただし，$r_k^{(i)}$，$q_k^{(j)}$ はベクトル $\boldsymbol{r}^{(i)}$，$\boldsymbol{p}^{(j)}$ の第 $k$ 成分を表します．

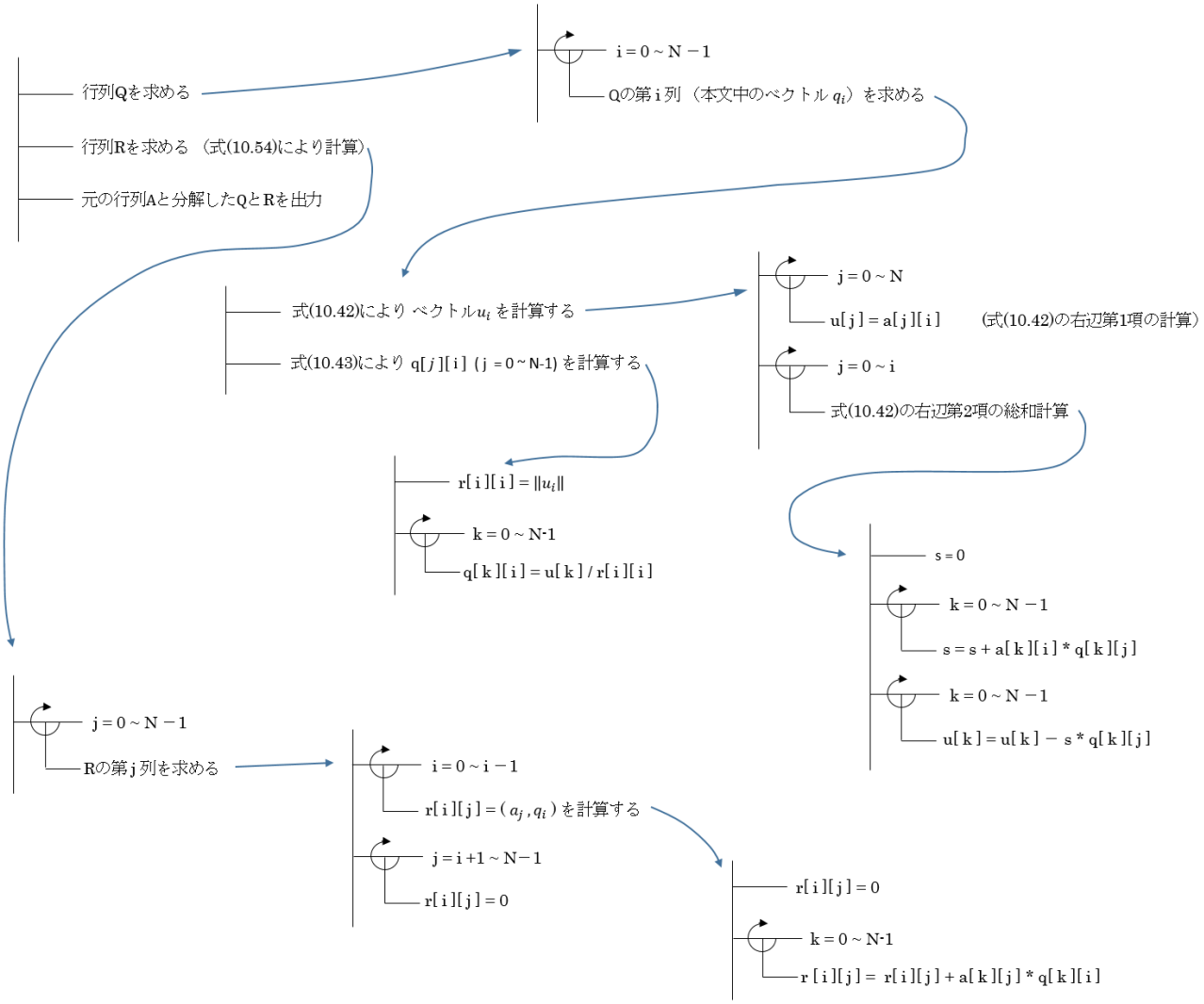
ここで $RQ = [s_{ij}]$ とすると，$i > j+1$ のとき

$$\begin{aligned} s_{ij} &= \sum_{k=1}^{n} r_k^{(i)} q_k^{(j)} \\ &= \sum_{k=1}^{j+1} r_k^{(i)} q_k^{(j)} + \sum_{k=j+2}^{n} r_k^{(i)} q_k^{(j)} \\ &= \sum_{k=1}^{j+1} 0 \cdot q_k^{(j)} + \sum_{k=j+2}^{n} r_k^{(i)} \cdot 0 \\ &= 0 \end{aligned} \tag{22}$$

となります．以上から $A_k$ に対して QR 法の反復計算を行った行列 $A_{k+1} = R_k Q_k$ もヘッセンベルグ行列となります．

### 問 10.2

SPD を下図に，プログラムを p10-2.cpp に示します．




**問 10.3**

プログラムを p10-3.cpp に示します．

## 第 11 章

**問 11.1**

プログラムを p11-1.cpp に示します．

**問 11.2**

三角関数の公式を用いて式変形すると以下のようになります．

$$
\begin{aligned}
T_2(x) &= \cos 2\theta \\
&= 2\cos^2\theta - 1 && \text{(倍角の公式 )} \\
&= 2x^{\,2} - 1 \\
T_3(x) &= \cos 3\theta \\
&= 4\cos^3\theta - 3\cos\theta && \text{(3 倍角の公式 )} \\
&= 4x^{\,3} - 3x \\
T_4(x) &= \cos 4\theta \\
&= \cos 2(2\theta) = 2\cos^2 2\theta - 1 && \text{(倍角の公式)} \\
&= 2\big(2\cos^2\theta - 1\big)^2 - 1 && \text{(倍角の公式)} \\
&= 2\big(4\cos^4\theta - 4\cos^2\theta + 1\big) - 1 \\
&= 8\cos^4\theta - 8\cos^2\theta + 1 \\
&= 8x^{\,4} - 8x^{\,2} + 1
\end{aligned}
$$

**問 11.3**

プログラムを p11-3.cpp に示します.

## 第 12 章

**問 12.1**

ルジャンドル多項式 $P_n(x)$ を用いた関数 $f\,(x)$ の補間は以下のようになります.

$$
f_n(x) = \sum_{i=0}^{n-1} c_i P_i(x) \tag{23}
$$

$$
c_k = \frac{2k+1}{2} \sum_{j=1}^{n} w_j f(x_j) P_k(x_j) \tag{24}
$$

ここで $x_j$ は $n$ 次ルジャンドル多項式 $P_n(x)$ の零点です．なお重み $w_j$（式 (12.15)）は，ルジャンドル多項式における $\mu_k$ 等の値を用いて整理すると以下のようになります.

まず式（11.33）より

$$
\mu_k = \frac{(2k)!}{2^k (k!)^2} \tag{25}
$$

となり，さらに式 (11.35) より $\lambda_n = \frac{2}{2n+1}$ を用いて

$$
\frac{\mu_n \lambda_{n-1}}{u_{n-1}} = \frac{2}{n} \tag{26}
$$

となります．これを式 (12.15) に代入すると次式を得ます.

$$
w_j = \frac{2}{n P_{n-1}(x_j) P'_n(x_j)} \tag{27}
$$

さらにルジャンドル多項式に関する公式

$$
P'_k(x) = \frac{k}{x^2 - 1}\left[x P_k(x) - k P_{k-1}(x)\right] \tag{28}
$$

において $k=n$，$x=x_j$ とすると

$$
\begin{aligned}
P_n'(x) &= \frac{n}{x_j^2-1}\left[xP_n(x_j)-nP_{n-1}(x)\right] \\
&= -\frac{n}{x_j^2-1}nP_{n-1}(x)
\end{aligned}
\tag{29}
$$

となり，これを先の (27) 式に代入すると

$$
w_j = \frac{2(1-x_j^2)}{n^2[P_{n-1}(x_j)]^2} \tag{30}
$$

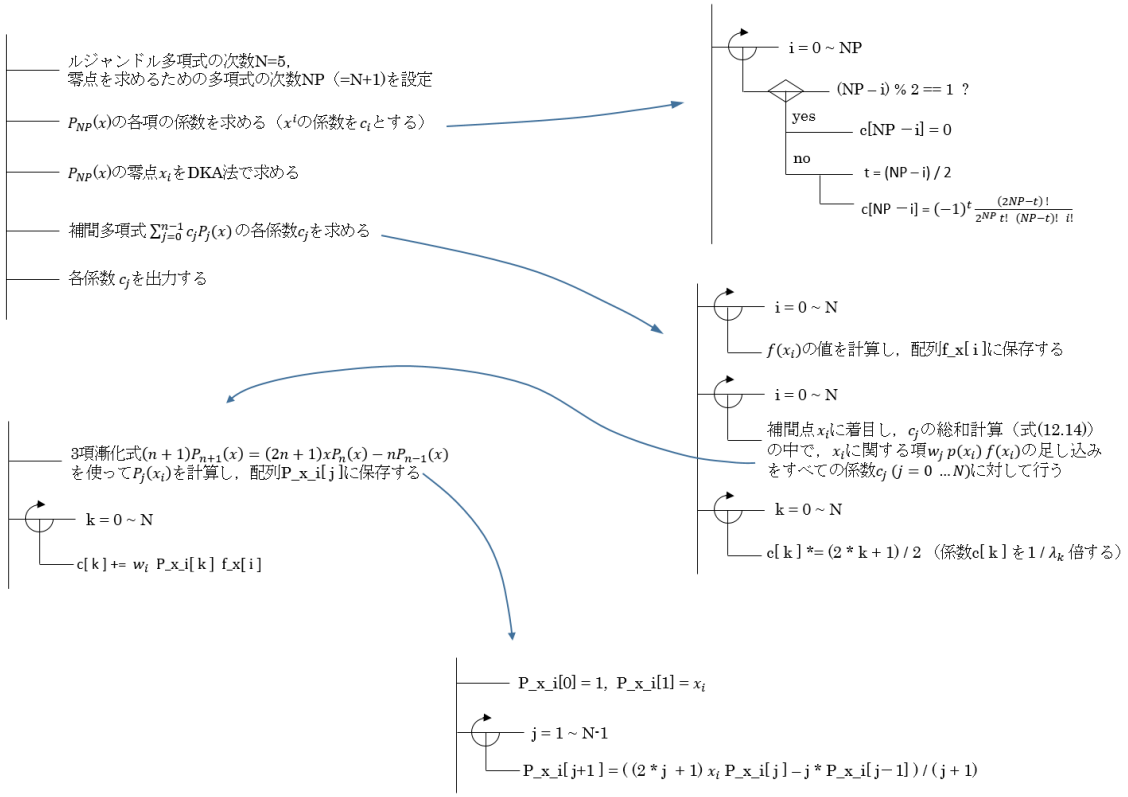
となります．

**問 12.2**

DKA 法を適用するのに必要となる $P_n$ の各項の係数は次式によって求めます．

$$
P_n(x) = \sum_{k=0}^{\lfloor n/2 \rfloor}(-1)^k \cdot \frac{(2n-2k)!}{2^n\,k!\,(n-k)!\,(n-2k)!} \cdot x^{n-2k} \tag{31}
$$

SPD を以下に，プログラムを p12-2.cpp に示します．



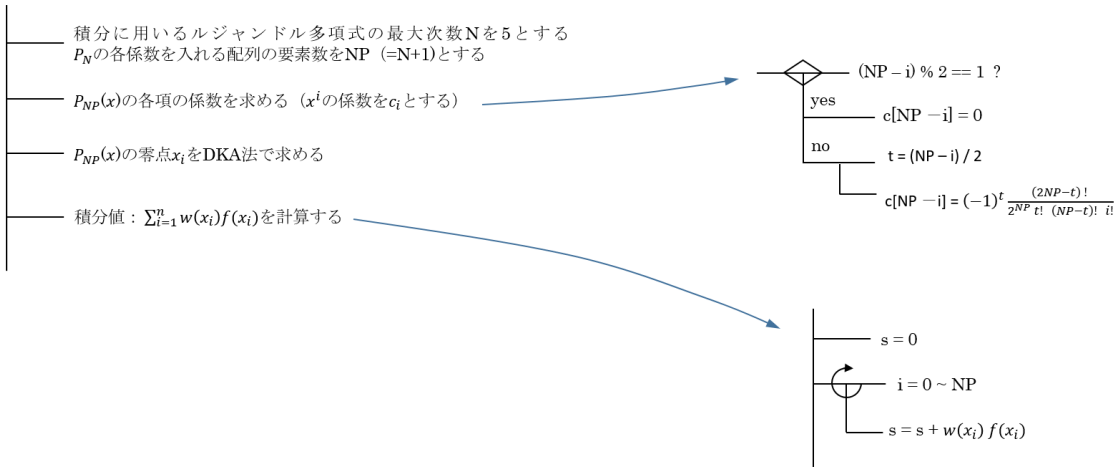

**問 12.3**

プログラムを p12-3.cpp に示します．なお，チェビシェフ多項式は式 (11.29) に示すように，べき多項式で表現できます．したがってチェビシェフ補間によって得られる補間関数と，式 (12.7) による補間関数とは同じになります．

## 第 13 章

### 問 13.1

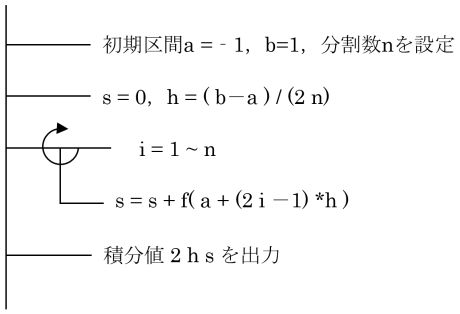
SPD を以下に，またプログラムを p13-1.cpp に示します．




### 問 13.2

【問題訂正】
正誤表に記載したように問題文中の「複合台形公式」を「複合中点公式」に訂正した上で解答を示します．

複合中点公式の SPD を以下に，またプログラムを p13-2.cpp に示します．

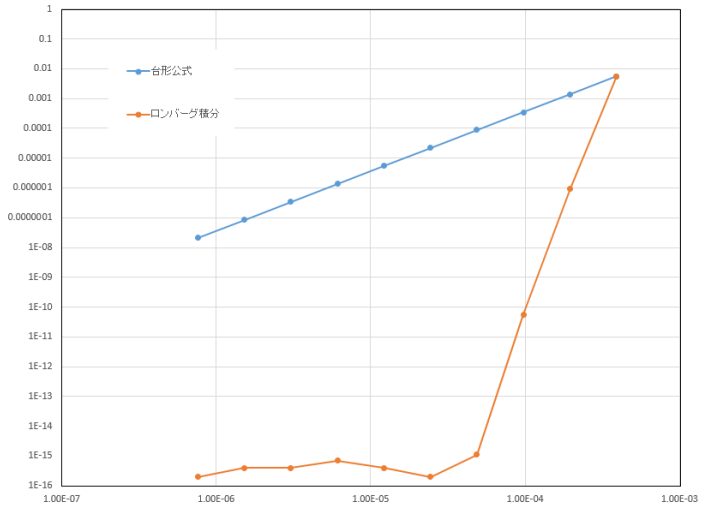



なおプログラムの実行結果は 1.6829700190 となります．ちなみに台形公式で計算すると 1.6828858712 となり，厳密な積分値 1.6829419696 と比較すると中点公式の方が精度が良くなっています．この点に関する詳しい説明は参考文献 [2] の 151 ページにあります．

### 問 13.3

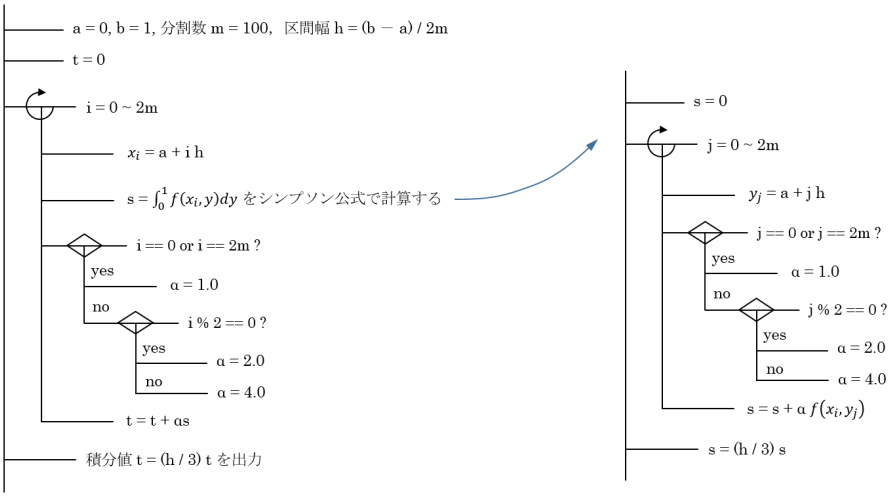
複合台形公式はロンバーグ積分の一部として含まれるため，ソースコード 13.3 の T 表を出力することで両手法の比較を行います．$i = 0, 1, \cdots, 9$ に対して，T[i][0]（台形公式）と T[i][i]（ロン

バーグ積分）を出力してグラフ化したものを以下に示します．台形公式に比べてロンバーグ積分は真の値への収束が早く，4 回目の反復で倍精度浮動数点数の限界付近まで精度がよくなっていることが分かります．




**問 13.4**

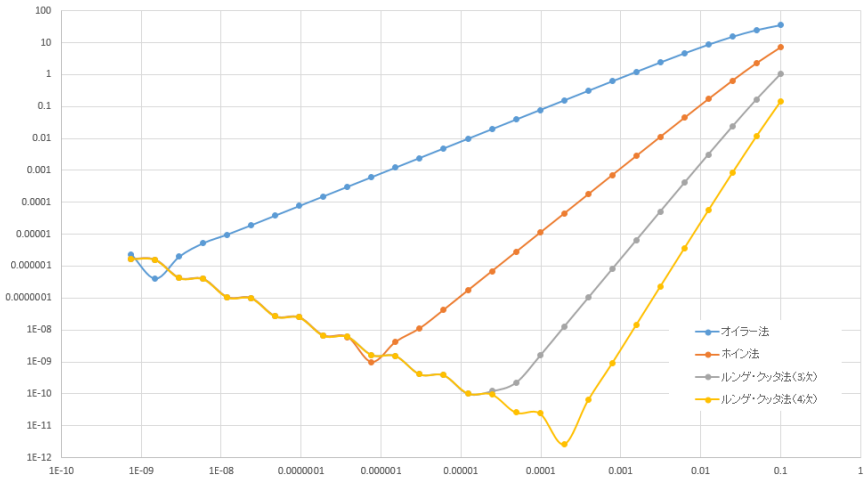
SPD を下図に，プログラムを p13-4.cpp に示します．




## 第 14 章

**問 14.1**

初期の分割幅を $h_0 = 0.1$ とした場合の誤差の変化を下図に示します．公式の次数が高いほど解の精度が良くなっているこが分かります．また 6 章の章末問題 6.1 と同様に，刻み幅を小さくしていくと途中から解の精度が悪化する様子も確認できます．




**問 14.2**

プログラムを p14-2.cpp に示します．

**問 14.3**

$y_{i+1}$ を $y_{i-1}$ でテイラー展開すると

$$y_{i+1} = y(x_i + 2h) = y_{i-1} + 2hy'(x_{i-1}) + 2h^2 y''(x_{i-1}) + O(h^3) \tag{32}$$

となります．また，リープフロッグ法の公式 (14.65)

$$Y_{i+1} = Y_{i-1} + 2hf(x_i, Y_i) \tag{33}$$

の右辺の $Y_{i-1}$，$Y_i$ を $y_{i-1}$，$y_i$ とおくと

$$\begin{aligned} y_{i-1} + 2hf(x_i, y_i) &= y_i + 2hy'(x_i) \\ &= y_{i-1} + 2h\{y'(x_{i-1}) + hy''(x_{i-1}) + O(h^2)\} \\ &= y_{i-1} + 2hy'(x_{i-1}) + 2h^2 y''(x_{i-1}) + O(h^3)\} \end{aligned} \tag{34}$$

を得ます．これらより局所離散化誤差 $\tau_{i+1}$ は

$$\begin{aligned} \tau_{i+1} &= y_{i+1} - \{y_{i-1} + 2hf(x_i, y_i)\} \\ &= \{y_{i-1} + 2hy'(x_{i-1}) + 2h^2 y''(x_{i-1}) + O(h^3)\} - \{y_{i-1} + 2hy'(x_{i-1}) + 2h^2 y''(x_{i-1}) + O(h^3)\} \\ &= O(h^3) \end{aligned} \tag{35}$$

となり，公式は 2 次となります．

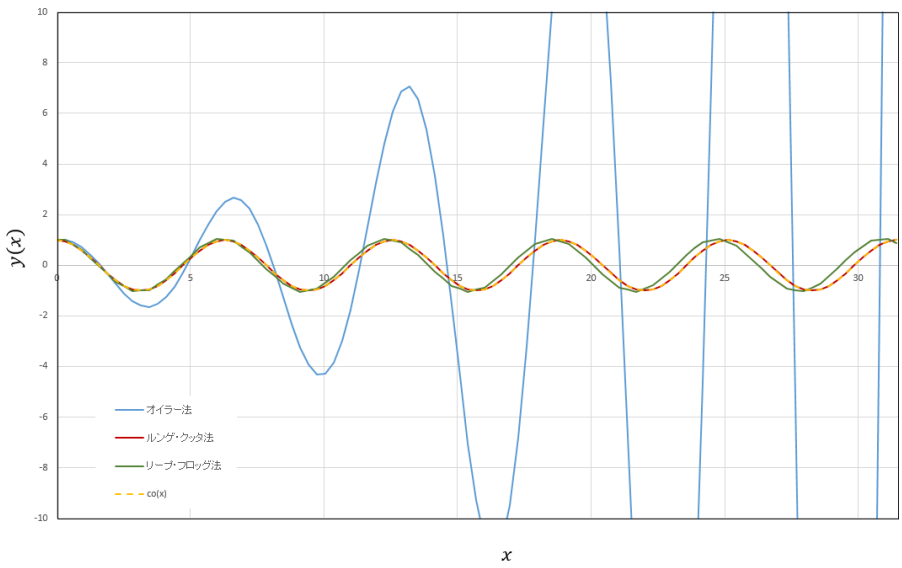
## 第 15 章

**問 15.1**

プログラムを p15-1.cpp に示します．

**問 15.2**

$x$ の範囲を 100 分割して解いた結果をグラフ化すると下図のようになります．4 次精度のルンゲ・クッタ法は解析解 ($y = \cos(x)$) とほとんど重なっています．公式の精度の順に従い，4 次のルンゲ・クッタ，2 次のリープ・フロッグ法，1 次のオイラー法の順で誤差が大きくなっています．




**問 15.3**

$y''$ と $y'$ をそれぞれ中心差分 (式 (15.86) と式 (15.90)) を用いて差分方程式を導出します．ここで得られる方程式は非線形となるため，Newton 法を適用して解を求めます．また初期値は境界の両端点を結ぶ一次関数 $f(x)$ を考え，分点における $f(x_i)$ の値を用います．プログラムを p15-3.cpp に示します．